

TASARIM DOKÜMANI · V1 · 2026-09-09

Suflör

Ekrandaki yazıyı yakalayıp tanıyan ve çeviren Windows masaüstü uygulaması. İki mod, dört katmanlı çeviri zekâsı, ücretsiz ve offline varsayılan.

İçindekiler

0. Özet

1. Amaç ve Kapsam

- 1.1 Problem
- 1.2 Hedef kullanıcı
- 1.3 Başarı kriterleri
- 1.4 v1 kapsam dışı (bilinçli olarak)

2. Ürün: İki Mod

- 2.1 Mod 1 — Snapshot Modu (anlık yakalama, seçmeli çeviri)
- 2.2 Mod 2 — Bölge İzleme Modu (dinamik, sürekli)
- 2.3 Kısayollar

3. Oyun-Özel Detaylar

- 3.1 Oyun profilleri
- 3.2 Oyun metni türleri ve OCR ön ayarları
- 3.3 Exclusive fullscreen
- 3.4 Anti-cheat duruşu
- 3.5 FPS etkisi

4. Çeviri Zekâsı: Dört Katman

- 4.1 Katman 0 — Terim Sözlüğü + Çeviri Hafızası (v1, en yüksek getiri)
- 4.2 Katman 1 — Yerel NMT (v1, Mod 2 varsayılanı)
- 4.3 Katman 2 — Yerel Küçük LLM (v1, Mod 1 varsayılanı)
- 4.4 Katman 3 — Kendi Modelinin İnce Ayarı (yol haritası, ayrı spec)
- 4.5 Bulut sağlayıcılar (v1, opsiyonel, varsayılan kapalı)
- 4.6 Motor seçim matrisi (varsayılan)

5. Runtime Mimari

- 5.1 Bileşen haritası
- 5.2 TextNormalizer neden birinci sınıf bileşen
- 5.3 Çekirdek veri modelleri
- 5.4 Veri akışı
- 5.5 Eşzamanlılık ve backpressure
- 5.6 Hata yönetimi
- 5.7 Performans bütçesi
- 5.8 Dizin yapısı

6. Teknoloji Seçimleri

7. Veri Saklama ve Gizlilik

8. Geliştirme Ajanları

- 8.1 Temel ilkeler
- 8.2 Orkestratör
- 8.3 Ajan kadrosu
- 8.4 Model atama gerekçesi

- 8.5 Ajanları birbirine bağlama protokolü
- 8.6 Görev paketi şablonu
- 8.7 Teslim raporu şablonu
- 8.8 İnşa dalgaları
- 8.9 Tüm ajanlar için ortak zorunlu kurallar

9. Test ve Kalite Stratejisi

10. Riskler ve Azaltımlar

11. Yol Haritası

12. Açık Sorular

Karar Kaydı

Tarih	2026-09-09
Durum	Onay bekliyor
Sürüm	v1 tasarımı
Platform	Windows 11 (x64)
Yığın	Python 3.12 + PySide6
Depo	efeyiit/suflor

Suflör: tiyatroda kuliste durup oyuncuya repliğini fısıldayan kişi. Uygulamanın tamamı bu metafor üzerine kurulu — oyunu kesmez, sahneyi bölmez, kenardan anlamı fısıldar. §5.6'daki "modal dialog asla açılmaz" kuralı bu isimden türeyen bir tasarım kısıtıdır, tersi değil.

0. Özet

Ekrandaki yazıyı yakalayıp tanıyan ve çeviren bir Windows masaüstü uygulaması. Oyun senaryosu birinci sınıf vatandaş, ancak her uygulamada çalışır.

İki mod:

- Snapshot Modu** — Kullanıcı kısayola basar, ekran donar, tespit edilen metin blokları çerçevelenir, kullanıcı istediklerini seçer, seçilenler bağlamli olarak çevrilir.
- Bölge İzleme Modu** — Kullanıcı ekranda bir dikdörtgen belirler; o alanda çıkan her yazı otomatik ve sürekli çevrilir, çeviri bölgeye yapışık yarı-şeffaf bir şeritte akar.

Üç temel duruş:

- Ücretsiz ve offline varsayılan.** İlk açılışta API anahtarı sorulmaz. Yerel OCR + yerel çeviri motorları indirilir, uygulama çalışır durumda gelir. Bulut sağlayıcı, isteyenin ayarlardan açtığı opsiyonel bir yükseltmedir.
- Anti-cheat açısından güvenli.** Hiçbir process injection, API hook veya memory okuma yok. Sadece Windows'un kendi ekran yakalama yolları.
- Hiçbir hata pipeline'ı durdurmaz.** Her hata bir sonraki tick'te toparlanma şansı bulur; oyun oynayan kullanıcıya modal dialog açılmaz.

1. Amaç ve Kapsam

1.1 Problem

Türkçe yaması olmayan oyunlar (özellikle JRPG, visual novel, strateji, indie yapımlar) diyalog ve menü metni yüzünden erişilemez kalıyor. Mevcut çözümler ya sadece kopyalanabilir metinle çalışıyor, ya oyun dosyalarına müdahale ediyor (ban riski), ya da her satır için ücretli API tüketiyor.

1.2 Hedef kullanıcı

Birincil: yabancı dilde oyun oynayan, İngilizce/Japonca metni anlamakta zorlanan oyuncu. Tek monitörden çoklu monitöre kadar farklı kurulumlar. Orta seviye donanım (entegre GPU dahil) desteklenmeli.

İkincil: yazılım arayüzü, hata mesajı, PDF, web sayfası çeviren genel kullanıcı.

1.3 Başarı kriterleri

Kriter	Hedef
Mod 2 uçtan uca gecikme (cache isabeti)	≤ 160 ms
Mod 2 uçtan uca gecikme (cache ıskası)	≤ 320 ms
Mod 1 seçim kutularının görünme süresi	≤ 250 ms
Oyun FPS düşüşü (Mod 2 aktif)	$\leq \%3$
İlk açılıştan ilk çeviriye	≤ 3 dakika (model indirme dahil)
Kullanıcının girmesi gereken API anahtarı	0
Aylık zorunlu maliyet	0 TL

1.4 v1 kapsam dışı (bilinçli olarak)

- **Yerinde kaplama (in-place overlay)** — çevirinin orijinal metnin tam üzerine binmesi. v2.
- **Exclusive fullscreen desteği** — v1 pencereli/borderless mod gerektirir ve bunu kullanıcıya açıkça söyler.
- **Katman 3: kendi çeviri modelini ince ayarlamak (LoRA)** — kendi spec'i ile ayrı alt proje, yol haritasında.
- TTS / sesli çeviri
- OCR modeli eğitimi veya ince ayarı
- macOS / Linux
- Hesap sistemi, bulut senkronizasyonu
- Manga/çizgi roman dikey metin okuma sırası mantığı
- Mobil / konsol

2. Ürün: İki Mod

2.1 Mod 1 — Snapshot Modu (anlık yakalama, seçmeli çeviri)

Akış:

1. Kullanıcı global kısayola basar (**Ctrl+Alt+T**).
2. O anki ekran **dondurulur** (aktif monitör; ayarla tüm monitörler).
3. Donmuş görüntü tam ekran, tıklanabilir bir katman olarak açılır.
4. **Yerel OCR anında** çalışır. Tespit edilen her metin bloğunun üstüne ince çerçeve çizilir — kullanıcı neyi seçebileceğini görür.

5. Kullanıcı seçim yapar: - Tek blok: tıklama - Çoklu blok: sürükleyerek lasso, veya **Ctrl** + tıklama - Hepsi: **Ctrl+A** - OCR'ın kaçırdığı yer: serbest dikdörtgen çizimi (o alan ayrıca OCR'lanır)
6. **Enter** → seçilen bloklar **tek istekte** çeviri motoruna gider. İstek hem görüntü kırpıntılarını hem OCR metnini taşır; vision destekli motor görsel bağlamı da görür (kim konuşuyor, menü mü diyalog mu).
7. Sonuç paneli: orijinal ↔ çeviri yan yana, kopyala butonu, "terim sözlüğüne ekle" butonu.
8. **Esc** her aşamada iptal.

Neden OCR önce yerel çalışıyor: Kullanıcının neyi seçebileceğini görmesi için anlık geri bildirim şart. Motoru beklemek 1-2 saniye boş ekran demek. Yerel OCR seçim kutularını verir, çeviri motoru kaliteli çeviriyi verir — bunlar farklı işler ve sıralı çalışırlar.

2.2 Mod 2 — Bölge İzleme Modu (dinamik, sürekli)

Akış:

1. Kullanıcı kısayola basar (**Ctrl+Alt+R**). Ekran karartılır, dikdörtgen çizer.
2. Bölge kalıcı hale gelir: kenarlarından yeniden boyutlandırılabilir, taşınabilir, profile kaydedilir.
3. İzleme döngüsü başlar; bölge her ~250 ms yakalanır (ayarlanabilir: 100–1000 ms).
4. **Değişim tespiti** — perceptual hash farkı. Değişmediyse hiçbir şey yapılmaz. CPU ve maliyet tam olarak burada kurtarılıyor: durağan bir diyalog ekranında pipeline neredeyse hiç çalışmaz.
5. **Debounce** — ~120 ms kararlılık bekleme (2 ardışık aynı hash). Metin harf harf yazılıyorsa yarısını okuyup çöp çeviri üretmemek için.
6. Yerel OCR → metin blokları → normalizasyon.
7. **Cache kontrolü.** Daha önce çevrildiyse anında ekrana. Oyunlarda aynı menü ve diyalog onlarca kez görünür; cache isabet oranı yüksek olacak.
8. Cache iskası → sözlük + çeviri hafızası enjekte edilir → çeviri motoru.
9. **Render:** bölgeye yapışık, yarı-şeffaf şerit. Kullanıcı şeridi koparıp ikinci monitöre atabilir.
10. Şerit son N çeviriyi kaydırılabilir geçmiş olarak tutar (varsayılan 20).

2.3 Kısayollar

Kısayol	İşlev
Ctrl+Alt+T	Snapshot modu
Ctrl+Alt+R	Bölge belirle / yeniden belirle
Ctrl+Alt+S	İzlemeyi başlat / durdur (toggle)
Ctrl+Alt+H	Şeridi gizle / göster
Ctrl+Alt+D	Şeridi kopar / yapıştır
Esc	Aktif overlay'i kapat

Hepsi ayarlardan değiştirilebilir. Çakışma durumunda uygulama uyarır ve kayıt yapmaz.

3. Oyun-Özel Detaylar

Mimari genel amaçlı, ancak oyun senaryosu birinci sınıf vatandaş. Bu bölüm oyunun getirdiği özel işleri tanımlar.

3.1 Oyun profilleri

Profil, bir oyun için tüm ayarların paketi:

```
Profile {
  id, name
  exe_name          # otomatik eşleştirme için
  window_title_pattern # regex, exe yetmezse
  regions[]         # birden fazla izleme bölgesi (diyalog + isim etiketi ayrı)
  source_lang, target_lang
  engine_preference  # nmt | llm | cloud
  tick_interval_ms
  glossary_id
  style_profile      # ton/kayıt talimatı
  strip_position     # below | above | detached + koordinat
  ocr_preset         # dialogue | menu | tooltip | subtitle
}
```

Uygulama aktif pencerenin exe adını izler; kayıtlı profil bulursa tek satırlık, kaybolan bir bildirim gösterir: "Elden Ring profili yüklensin mi?" Otomatik yükleme ayarla açılabilir.

Profiller `.json` olarak dışa/içe aktarılabilir — topluluk profil paylaşabilir. Bu, uygulamanın en güçlü ağ etkisi: bir kullanıcının hazırladığı "Persona 5 profili + terim sözlüğü" herkese yarar.

3.2 Oyun metni türleri ve OCR ön ayarları

Oyun metni tek tip değil; her tür farklı işlem gerektiriyor:

Tür	Özellik	Gereken işlem
Diyalog kutusu	Sabit konum, harf harf yazılır, kişi adı ayrı	Debounce kritik; isim etiketi ayrı bölge olarak izlenmeli
Menü / envanter	Kısa, çok sayıda, tabular	Blok gruplama kapatılmalı; her öge ayrı segment
Eşya/yetenek açıklaması	Uzun, sayı ve değişken içerir	Sayılar ve yüzdeler korunmalı; terim sözlüğü kritik
Altyazı (sinematik)	Alt orta, hızlı değişir	Kısa tick, agresif debounce
HUD / uyarı	Anlık, kaybolan	İzlenmemeli (gürültü); istenirse ayrı bölge
Stilize başlık	Süslü font, gölge, doku	OCR güveni düşer; kullanıcı Mod 1 ile manuel çözmeli

`ocr_preset` bu türleri yansıtır. Ön ayar; blok gruplama, güven eşiği, ölçekleme faktörü ve kontrast ön işlemlerini değiştirir.

3.3 Exclusive fullscreen

Overlay penceresi exclusive fullscreen'de görünmez. Bu Windows'un davranışı; aşılması için injection gerekir, ki bunu yapmıyoruz.

Davranış: uygulama, izlenen pencerenin exclusive fullscreen olup olmadığını tespit eder (DWM durumu). Öyleyse **açıkça** söyle:

"Bu oyun exclusive fullscreen modunda. Çeviri şeridi görünmeyecek. Oyunun grafik ayarlarından **Borderless / Windowed Fullscreen** seçin — ya da şeridi **Ctrl+Alt+D** ile koparıp ikinci monitöre alın."

Sessizce çalışmıyormuş gibi görünmek en kötü seçenek. Kullanıcı ne yapması gerektiğini bilmeli.

3.4 Anti-cheat duruşu

Bu bir kısıt değil, bilinçli mimari duruş: **oyun hesabının banlanma riski sıfır olmalı**.

Yapılmayacaklar: process injection, DLL enjeksiyonu, API hook (Detours vb.), memory okuma/yazma, oyun dosyası değiştirme, Direct3D present hook.

Yapılacaklar: **mss** / BitBlt / DXGI Desktop Duplication ile ekran yakalama, **RegisterHotKey** ile global kısayol, **WS_EX_LAYERED | WS_EX_TRANSPARENT | WS_EX_TOPMOST** ile normal bir üst-katman pencere.

Bu, OBS'in ve Discord overlay'inin *daha azını* yapan bir kümedir. Anti-cheat açısından güvenli sınıf.

3.5 FPS etkisi

Hedef: $\leq$ %3 FPS düşüşü. Bunu sağlayan tasarım kararları:

- Değişim tespiti sayesinde durağan sahnede OCR hiç çalışmaz.
- Sadece bölge yakalanır, tüm ekran değil (küçük bölge = küçük bellek trafiği).
- OCR CPU'da (ONNX) çalışır; GPU'yu oyuna bırakır. Kullanıcı isterse GPU'ya alabilir.
- Overlay penceresi statik; sadece çeviri değiştiğinde yeniden çizilir. Animasyon yok.
- Yerel LLM (Katman 2) Mod 2'de **varsayılan olarak kapalı** — VRAM'i oyuna paylaşmak FPS'i düşürür. Kullanıcı bilinçli açar.

4. Çeviri Zekâsı: Dört Katman

OCR baştan beri ücretsiz ve yereldir (ONNX). Maliyet ve kalite sorusu yalnızca **çeviri motoru** ile ilgilidir. Motorlar dört katman halinde tasarlanmıştır; hepsi aynı **TranslationProvider** arayüzünü uygular, dolayısıyla motor değiştirmek bir config değişikliğidir.

Sıfırdan model eğitmek (pre-training) kapsam dışıdır: milyonlarca dolar GPU zamanı ve milyarlarca cümlelik veri gerektirir. Buna gerek de yok — aşağıdaki katmanlar aynı sonuca çok daha az maliyetle ulaşır.

4.1 Katman 0 — Terim Sözlüğü + Çeviri Hafızası (*v1, en yüksek getiri*)

Tamamen ücretsiz, sıfır eğitim, ve tam olarak "uygulamaya özel" olan katman.

Terim sözlüğü (glossary): oyun başına karakter isimleri, eşya/yetenek/yer adları, hitap ekleri, çevrilmemesi gereken özel adlar. Motora zorunlu kısıt olarak enjekte edilir.

Üslup profili (style profile): serbest metin talimat — "karanlık fantezi JRPG, resmi dil, siz kullan, küfür yumuşatılmasın". Yalnızca LLM motorlarında etkilidir.

Çeviri hafızası (TM): daha önce çevrilmiş her segment saklanır. Yeni segment geldiğinde normalize edilmiş metin üzerinde fuzzy eşleşme (rapidfuzz, trigram) yapılır; en yakın k örnek motora few-shot örnek olarak verilir. Tutarlılık burada doğar: aynı yetenek adı ikinci kez farklı çevrilmez.

Bu katmanın etkisi hafife alınamaz. Vasat bir motoru bile o oyuna özel hissettiren şey budur ve maliyeti sadece koddur.

4.2 Katman 1 — Yerel NMT (v1, Mod 2 varsayılanı)

CTranslate2 üzerinde int8 quantize edilmiş nöral çeviri modeli:

- **NLLB-200-distilled-600M** — 200 dil, int8 sonrası ~600 MB
- veya **Helsinki-NLP OPUS-MT** dil çifti başına küçük modeller (~100-300 MB), daha hızlı, dil çifti başına indirme

Performans: CPU'da satır başına ~50-150 ms. Sıfır maliyet, internetsiz çalışır, GPU gerekmez.

Dürüst zayıf yönü: cümle cümle çevirir, geniş bağlamı görmez, oyun argosu ve deyimlerde tökezler. Katman 0 ile birlikte belirgin şekilde düzelir ama LLM seviyesine çıkmaz. Mod 2'nin sürekli döngüsü için doğru araç: hızlı ve bedava.

4.3 Katman 2 — Yerel Küçük LLM (v1, Mod 1 varsayılanı)

llama.cpp üzerinde q4 quantize edilmiş 4B sınıfı çok dilli instruct modeli (Gemma 3 4B veya Qwen3 4B sınıfı):

- İndirme ~2.5-3 GB, çalışma ~4-6 GB VRAM (CPU fallback mümkün ama yavaş)
- Bağlamı anlar, talimat dinler, terim sözlüğünü gerçekten uygular
- 4B ve üstü Gemma 3 sürümleri **görüntü de görür** → Mod 1'in vision avantajı bulut olmadan elde edilir
- Satır başına ~0.5-2 s (streaming ile kullanıcı beklemeyi hissetmez)

Mod 1 için ideal (kullanıcı tetikler, seyrek, kalite önemli). Mod 2 için varsayılan kapalı; güçlü makinede kullanıcı açabilir.

Model seçimi kurulum sırasında doğrulanacak: aday modeller TR çıktı kalitesi ve hız açısından altın set üzerinde ölçülüp seçilir (bkz. Bölüm 12).

4.4 Katman 3 — Kendi Modelinin İnce Ayarı (yol haritası, ayrı spec)

"Uygulamaya özel çeviri AI"nın tam karşılığı. 4B sınıfı açık bir model, oyun diyalogu üzerinde QLoRA ile ince ayarlanır.

Veri kaynakları:

- **OpenSubtitles TR-EN korpusu** — on milyonlarca konuşma dili cümle çifti. Oyun diyaloguna şaşırtıcı derecede yakın, açık ve ücretsiz. En büyük kaldıraç.
- **Türkçe yama topluluğu** — Türkiye'deki oyun çevirisi sahnesi büyük; mevcut yamalardan hizalanmış çift dilli veri çıkarılabilir. **Lisans dikkati gerektirir**; izinsiz kullanılmaz.
- **Damıtma (distillation)** — güçlü bir bulut modele iyi prompt'larla bir oyun metni korpusu çevirtip, küçük yerel modeli o çıktılar üzerinde eğitmek. Dar bir alanda küçük modeli kendi ağırlığının çok üstüne çıkaran, kanıtlanmış yöntem.

Maliyet: Unsloth ile 8 GB VRAM'de 4B QLoRA döner; ya da bulut GPU birkaç saat kiralanır — onlarca dolar mertebesi. Sonuç kullanıcı için tamamen ücretsiz ve offline.

Neden v1'de değil: veri toplama + hizalama + eğitim + kalite ölçüm koşumu kendi başına bir proje. v1'i buna bağlarsak v1 aylarca çıkmaz. Pluggable sağlayıcı katmanı sayesinde geldiğinde yalnızca yeni bir **FineTunedProvider** olur; mimari değişikliği gerektirmez.

4.5 Bulut sağlayıcılar (v1, opsiyonel, varsayılan kapalı)

Ayarlardan açılır. Kullanıcı kendi API anahtarını girer; anahtar Windows DPAPI ile şifreli saklanır, log'a asla yazılmaz.

Desteklenecekler: Claude (vision), Gemini (vision), DeepL (metin). Hepsi aynı arayüz.

Kota/ağ hatasında otomatik olarak yerel motora düşülür ve kullanıcı durum çubuğunda bilgilendirilir.

4.6 Motor seçim matrisi (varsayılan)

	Mod 1 (Snapshot)	Mod 2 (Canlı)
OCR	Yerel ONNX	Yerel ONNX
Çeviri	Katman 2 (yerel LLM, vision)	Katman 1 (yerel NMT)
Katman 0	Aktif	Aktif
Bulut	Kapalı (opsiyonel)	Kapalı (opsiyonel)
Düşüş zinciri	LLM → NMT	NMT → hata bandı

5. Runtime Mimari

5.1 Bileşen haritası

Her bileşen tek bir iş yapıyor, arayüzü üzerinden konuşuyor, tek başına test edilebiliyor.

#	Bileşen	Sorumluluk	Arayüz (özet)
1	<code>CaptureService</code>	Ekran / bölge yakalama, monitör ve DPI farkındalığı	<code>capture_full(monitor)</code> , <code>capture_region(rect) -> Frame</code>
2	<code>ChangeDetector</code>	Frame değişti mi, kararlı mı	<code>has_changed(frame) -> bool</code>
3	<code>OcrEngine</code> (arayüz)	Görüntü → metin blokları	<code>recognize(frame, preset) -> list[TextBlock]</code>
4	<code>TextNormalizer</code>	OCR çıktısını temiz segmentlere çevirme	<code>normalize(blocks, preset) -> list[Segment]</code>
5	<code>GlossaryStore</code>	Oyuna özel terim sözlüğü	<code>lookup(text) -> list[TermHit]</code>
6	<code>TranslationMemory</code>	Geçmiş çevirilerden fuzzy örnek	<code>find_similar(text, k)</code> , <code>add(pair)</code>
7	<code>TranslationProvider</code> (arayüz)	Asıl çeviri	<code>translate(req) -> TranslationResult</code>
8	<code>TranslationCache</code>	Bellek LRU + SQLite kalıcı	<code>get(key)</code> , <code>put(key, val)</code>
9	<code>OverlayRenderers</code>	Yapışık şerit / koparılabılır pencere	<code>render(segments)</code> , <code>set_status(msg)</code>
10	<code>ProfileStore</code>	Oyun profilleri	<code>load(exe_name)</code> , <code>save(profile)</code> , <code>export/import</code>
11	<code>PipelineOrchestrator</code>	Aşamaları kuyruklarla bağlama, backpressure	<code>start(mode, profile)</code> , <code>stop()</code>
12	<code>HotkeyService</code>	Global kısayollar	<code>register(combo, handler)</code>
13	<code>ModelManager</code>	Model indirme, checksum doğrulama, sürüm	<code>ensure(model_id)</code> , <code>progress_signal</code>
14	<code>SettingsStore</code> + <code>SecretStore</code>	Ayarlar; DPAPI ile şifreli anahtar	<code>get/set</code> , <code>put_secret/get_secret</code>

`TranslationProvider` uygulamaları: `LocalNmtProvider` · `LocalLlmProvider` · `CloudProvider` · (ileride) `FineTunedProvider` · `FakeProvider` (test)

`OcrEngine` uygulamaları: `RapidOcrEngine` (v1 varsayılan, ONNX) · `PaddleOcrEngine` (opsiyonel) · `FakeOcrEngine` (test)

5.2 `TextNormalizer` neden birinci sınıf bileşen

Çeviri kalitesinin yarısı burada belirlenir. OCR iki satıra bölünmüş bir cümle verir; bunu iki ayrı cümle olarak çevirirsek sonuç bozuk çıkar.

Sorumlulukları:

- Satır birleştirme (aynı paragrafa ait satırlar), hyphen/kesme çizgisi çözme
- Güven eşiği altındaki blokları düşürme
- Tek karakterlik gürültü ve süs karakterlerini atma
- Komşu blokları aynı diyalog kutusuna gruplama (`dialogue` ön ayarı) veya gruplamama (`menu` ön ayarı)
- Konuşan kişi etiketini metinden ayırma
- Sayı, yüzde ve değişken yer tutucularını (`{0}`, `%s`, renk etiketleri) koruma işaretleriyle sarma

Hepsi saf fonksiyon → altın görüntü setiyle bağımsız ve hızlı test edilebilir. Bu bileşen ayrı tutulmazsa OCR ve çeviri kodu içine sızar ve test edilemez hale gelir.

5.3 Çekirdek veri modelleri

Bunlar sözleşme dosyalarıdır. Tüm bileşenler bunlara bağımlıdır; yalnızca Sözleşme Ajanı (A1) değiştirebilir.

```
@dataclass(frozen=True)
class Rect:
    x: int; y: int; w: int; h: int
    monitor_index: int
    dpi_scale: float

@dataclass(frozen=True)
class Frame:
    image: np.ndarray          # BGR, uint8
    rect: Rect
    captured_at: float        # monotonic
    seq: int

@dataclass(frozen=True)
class TextBlock:
    text: str
    bbox: Rect
    confidence: float
    line_boxes: tuple[Rect, ...]

@dataclass(frozen=True)
class Segment:
    text: str                  # normalize edilmiş
    bbox: Rect
    speaker: str | None
    placeholders: tuple[str, ...] # korunacak yer tutucular
    source_blocks: tuple[int, ...]

@dataclass(frozen=True)
class TranslationRequest:
    segments: tuple[Segment, ...]
    source_lang: str | None      # None = otomatik algıla
    target_lang: str
    glossary_hits: tuple[TermHit, ...]
    tm_examples: tuple[Pair, ...]
    style_profile: str | None
    image_crops: tuple[np.ndarray, ...] | None # vision motorlari için

@dataclass(frozen=True)
class TranslationResult:
    translations: tuple[str, ...] # segments ile birebir hizali
    provider_id: str
    latency_ms: float
    from_cache: bool
    detected_lang: str | None
    partial: bool = False        # streaming ara sonucu
```

Hata taksonomisi (sözleşmenin parçası):

```
class TranslatorError(Exception): ...
class CaptureError(TranslatorError): ... # bolge gecersiz, monitor yok
class ModelMissingError(TranslatorError): ... # indirme gerekli
class OcrError(TranslatorError): ...
class ProviderUnavailable(TranslatorError): ... # OOM, ag, kota
class ProviderTimeout(TranslatorError): ...
class ContractViolation(TranslatorError): ... # segment/ceviri sayisi uyusmadi
```

5.4 Veri akışı

Mod 2 (canlı):

```
QTimer(250ms)
-> CaptureService.capture_region()           [worker thread]
-> ChangeDetector.has_changed() --hayir-->   DUR, frame'i dus
-> debounce: 2 ardisik kararli hash bekle
-> OcrEngine.recognize()                     [worker; ONNX GIL'i birakir]
-> TextNormalizer.normalize()
-> TranslationCache.get() --isabet-->        Renderer (~0 ms)
-> GlossaryStore.lookup() + TM.find_similar()
-> TranslationProvider.translate()          [worker]
-> Cache.put() + TM.add()
-> Qt signal -> OverlayRenderer.render()    [UI thread]
```

Mod 1 (snapshot):

```
Hotkey -> capture_full() -> SnapshotWindow (donmus goruntu)
-> OcrEngine.recognize() -> secim kutularini ciz
==> KULLANICI SECIM YAPAR <==
Enter -> secili bloklar + goruntu kirpintilari
-> TextNormalizer -> Glossary + TM
-> LocalLlmProvider.translate(image_crops=...) [streaming]
-> sonuc panelinde akarak yazilir
```

İki mod aynı bileşenleri farklı grafikte bağlar. Ortak olmayan tek şey giriş tetikleyicisi ve çıkış yüzeyi.

5.5 Eşzamanlılık ve backpressure

Tek process, beş aşamalı pipeline. UI thread'i yalnızca sinyal alır ve çizer; hiçbir bloklayıcı iş yapmaz.

- Ağır aşamalar (`capture`, `ocr`, `translate`) `QThreadPool` worker'larında koşar. ONNX Runtime ve CTranslate2 inference sırasında GIL'i bıraktığı için bu gerçek paralellik sağlar.
- Aşamalar arası `queue.Queue(maxsize=1)`.
- Backpressure kuralı:** yeni frame geldiğinde işlenmemiş eski frame **düşürülür**, biriktirilmez. Canlı çeviride doğru davranış "hepsini sırayla işle" değil, "en güncel olanı göster" — kullanıcı 3 saniye önceki diyalogun çevirisini görmek istemiyor.
- Her frame bir `seq` numarası taşır. Render aşaması, `seq` küçükse sonucu **yok sayar** (geç gelen eski çeviri ekrana basılmaz).
- İptal: kullanıcı izlemeyi durdurduğunda veya bölge değiştiğinde uçuşta olan istekler iptal token'ı ile iptal edilir.

Motoru ayrı process'e taşıma dikişi: Aşama sınırları temiz arayüzler olarak çizilir. Gelecekte motor ayrı process'e taşınmak istenirse `PipelineOrchestrator`'ın kuyruk implementasyonu değişir, aşama kodları değişmez. v1'de bu dikiş **çizilir ama kullanılmaz** — IPC karmaşıklığının bedeli bugün ödenmez.

5.6 Hata yönetimi

Aşama	Hata	Davranış
Capture	Monitör/DPI değişti, pencere kapandı	Bölgeyi yeniden doğru; "bölge geçersiz" bandı, izleme duraklat
Capture	Yakalama başarısız (korumalı içerik)	3 deneme sonrası net mesaj: bu pencere yakalanamıyor
OCR	Model yüklenemedi	Açılışta preflight kontrolü + tek tıkla indirme
OCR	Düşük güven / boş sonuç	Sessizce atla, şeride dokunma (titreme yapma)
Normalizer	Segment üretilemedi	Atla, sayaç artır; 10 ardışık boşa ön ayar önerisi göster
Provider	Yerel model OOM	Otomatik NMT'ye düş, durum çubuğunda bilgilendir
Provider	Timeout (>3 s)	İsteği iptal et, bekleme göstergesi, sonraki frame'i bekle
Provider	Ağ / kota hatası (bulut)	Exponential backoff → yerel motora otomatik düşüş
Provider	Segment/çeviri sayısı uyuşmadı	ContractViolation ; sonucu at, tek tek yeniden dene
Render	Monitör çıkarıldı	Şeridi birincil monitöre taşı
Store	SQLite kilidi / bozulma	WAL modu; bozulmada cache'i yeniden oluştur (TM'yi asla silme)

Genel ilke: hiçbir hata pipeline'ı durdurmaz; her hata bir sonraki tick'te toparlanma şansı bulur. Kalıcı hatalar durum çubuğunda tek satır olarak görünür — **modal dialog asla açılmaz**. Oyun oynayan birinin ekranına modal dialog atmak affedilemez.

5.7 Performans bütçesi

Mod 2, aşama bazında:

Aşama	Bütçe	Cache isabeti	Cache ıskası
Capture (bölge)	≤ 10 ms	✓	✓
Değişim tespiti	≤ 5 ms	✓	✓
OCR (ONNX, ~600 px bölge)	≤ 120 ms	✓	✓
Normalizasyon	≤ 5 ms	✓	✓
Cache araması	≤ 1 ms	✓	✓
Sözlük + TM araması	≤ 10 ms	—	✓
Yerel NMT çevirisi	≤ 150 ms	—	✓
Render	≤ 16 ms (1 kare)	✓	✓
Uçtan uca toplam		~157 ms	~317 ms

Bu, §1.3'teki ≤ 160 ms / ≤ 320 ms hedeflerinin bütçe dökümüdür.

Bu bütçe temenni değil: her aşama süresini histogram olarak kaydeder, performans testi bütçeyi aşan aşamada **kırılır**. Bütçe aşımı bir bug'dır, bir gözlem değil.

5.8 Dizin yapısı

Dizin sınırları aynı zamanda **ajan sahiplik sınırlarıdır** (Bölüm 8).

```
src/
  contracts/          # A1 - veri modelleri, arayuzler, hata taksonomisi
    models.py  interfaces.py  errors.py
  capture/            # A2
    service.py  change_detector.py  monitors.py  dpi.py
  ocr/                 # A3
    engine.py  rapidocr_engine.py  normalizer.py  presets.py
  translate/           # A4
    provider.py  local_nmt.py  local_llm.py  cloud.py  prompts.py
  store/               # A5
    db.py  cache.py  memory.py  glossary.py  profiles.py  migrations/
  pipeline/            # A6
    orchestrator.py  stages.py  queues.py  metrics.py
  ui/                  # A7
    app.py  snapshot_window.py  region_selector.py
    overlay_strip.py  settings_dialog.py  tray.py  hotkeys.py
  models_mgr/          # A8
    manager.py  registry.py  download.py
  tests/               # A9
    fixtures/golden/  unit/  integration/  perf/
  packaging/           # A8
    app.spec  build.ps1  LICENSES/
  docs/
    superpowers/specs/  user-guide.md
```

6. Teknoloji Seçimleri

Alan	Seçim	Gerekçe
Dil	Python 3.12	En zengin OCR/AI ekosistemi; tek dil = ajan sürtünmesi minimum
UI	PySide6 (Qt 6)	Olgun transparan/topmost pencere desteği, LGPL, iyi DPI
Yakalama	<code>mss</code> (v1)	Basit, hızlı, saf Python. DXGI Desktop Duplication v2 için değerlendirilecek
OCR	RapidOCR (ONNX Runtime)	Yerel, ücretsiz, hafif, çok dilli
NMT	CTranslate2 + NLLB-200-distilled-600M (int8)	Hızlı CPU çıkarımı, küçük ayak izi, 200 dil
Yerel LLM	llama.cpp (<code>llama-cpp-python</code>), 4B q4 GGUF	Vision destekli, talimat dinler, tüketici GPU'da çalışır
Fuzzy eşleşme	rapidfuzz	C++ hızında, TM için ideal
Veri	SQLite (WAL)	Tek dosya, sıfır kurulum; cache + TM + sözlük + profil
Şifreleme	Windows DPAPI (<code>pywin32</code>)	API anahtarları için; işletim sistemi güvencesi
Paketleme	PyInstaller (one-dir)	Modeller ayrı indirilir → exe küçük kalır
Test	pytest + pytest-qt + pytest-benchmark	Qt sinyalleri ve performans bütçesi testi
Log	<code>structlog</code> → dosya + ring buffer	Yapılandırılmış log; anahtar/metin asla loglanmaz

Bağımlılık disiplini: yeni bağımlılık eklemek onay gerektirir. Kriterler: lisans (GPL kabul edilmez; LGPL/MIT/Apache tercih), paket boyutu, bakım durumu, PyInstaller uyumu.

7. Veri Saklama ve Gizlilik

Konum: `%LOCALAPPDATA%/Suflor/` (dizin adı ASCII; Türkçe karakter yol sorunlarına yol açabilir)

```
db.sqlite      # cache, TM, sozluk, profiler
settings.json  # ayarlar (anahtar YOK)
secrets.dat    # DPAPI ile sifreli API anahtarları
models/       # indirilen OCR/NMT/LLM modelleri
logs/         # donen (rotating) log dosyaları
```

Gizlilik durumu:

- Varsayılan yapılandırmada **hiçbir veri cihazdan çıkmaz**.
- Ekran görüntüleri diske yazılmaz; yalnızca bellekte tutulur ve işlendikten sonra bırakılır. Tek istisna: kullanıcının açıkça kaydettiği snapshot.
- Log'a OCR metni, çeviri içeriği veya API anahtarı **yazılmaz**; yalnızca metrik, sayaç ve hata tipi.
- Bulut sağlayıcı açıldığında kullanıcıya tek seferlik ve net bir bilgilendirme gösterilir: hangi veri hangi sağlayıcıya gidiyor.

- Telemetri yok.

8. Geliştirme Ajanları

Bu bölüm, uygulamayı **inşa edecek** yapay zekâ ajanlarını tanımlar. Bunlar uygulamanın içinde çalışan bileşenler değildir; geliştirme sürecinde çalışan Claude Code alt-ajanlarıdır (subagent).

8.1 Temel ilkeler

Dört ilke, çok ajanlı geliştirmenin işe yarayıp yaramayacağını belirler:

1. **Sözleşme önce (contract-first)**. Paralel çalışmanın tek şartı, herkesin aynı arayüzlere yazmasıdır. `src/contracts/` önce yazılır, dondurulur ve başka hiçbir ajan dokunamaz.
2. **Dosya sahipliği (file ownership)**. Her ajanın sahip olduğu izin var; başka dizine yazamaz. Bu, çakışmayı imkânsız hale getirir — kilit veya koordinasyon gerekmez.
3. **Kanıtızsız iddia yok**. Bir ajan "testler geçti" diyemez; komutu çalıştırıp çıktısını teslim raporuna yapıştırması gerekir. Kanıt yoksa iş bitmemiş sayılır.
4. **Model, hatanın maliyetine göre seçilir**. Geri dönüşü pahalı işler (sözleşme tasarımı, eşzamanlılık, inceleme) güçlü modele; sınırları net ve testli implementasyon işleri hızlı modele; mekanik üretim en hafif modele.

8.2 Orkestratör

Rol	Teknik lider / orkestratör
Model	Opus 5
Nerede çalışır	Ana oturum (subagent değil)
Sahiplik	Plan, dalga sıralaması, sözleşme değişiklik kararları, entegrasyon, birleştirme
Yazmadığı şey	Modül kodu. Yalnızca sözleşme değişikliklerini ve entegrasyon yapıştırmasını yazar.

Sorumlulukları:

- İnşa dalgalarını sıraya koyar ve her dalgada hangi ajanların paralel çalışacağına karar verir
- Her ajana **görev paketi** (§8.6) hazırlar
- Teslim raporlarını (§8.7) okur, Reviewer'in verdiği kararı uygular: kabul veya düzeltme paketi
- Sözleşme değişikliği taleplerini tek elden değerlendirir — bir sözleşme değişikliği, ona bağlı tüm ajanları etkilediği için asla bir worker tarafından tek başına yapılamaz
- Dalga sonunda entegrasyon testini **kendisi** çalıştırır; hiçbir worker'ın "bende çalışıyor" iddiasına dayanmaz

Neden Opus 5: bu rolde verilen her karar ondan sonraki bütün işi şekillendiriyor. Yanlış bir sözleşme kararı beş ajanın işini çöpe atar.

8.3 Ajan kadrosu

Kod	Ajan	Model	Sahip olduğu dizin	Bağımlı olduğu
A1	Sözleşme ve Çekirdek Tipler	Opus 5	<code>src/contracts/</code>	—
A2	Yakalama ve Ekran	Sonnet 5	<code>src/capture/</code>	A1
A3	OCR ve Normalizasyon	Sonnet 5	<code>src/ocr/</code>	A1
A4	Çeviri Motorları	Sonnet 5	<code>src/translate/</code>	A1, A5
A5	Kalıcılık ve Hafıza	Sonnet 5	<code>src/store/</code>	A1
A6	Pipeline ve Eşzamanlılık	Opus 5	<code>src/pipeline/</code>	A1, A2, A3, A5 + A4'ün sözleşmesi
A7	UI ve Overlay	Sonnet 5	<code>src/ui/</code>	A1, A6
A8	Paketleme ve Model Dağıtım	Sonnet 5	<code>src/models_mgr/</code> , <code>packaging/</code>	A1
A9	Test Altyapısı ve Fixture	Sonnet 5 / Haiku 4.5	<code>tests/</code>	A1
A10	İnceleme (Reviewer)	Opus 5	(yazmaz — salt okuma)	tümü
A11	Dokümantasyon ve Yerelleştirme	Haiku 4.5	<code>docs/</code> , <code>src/ui/i18n/</code>	tümü

A1 — Sözleşme ve Çekirdek Tipler Ajansı

Model	Opus 5
Dizin	<code>src/contracts/</code>
Ne zaman	Dalga 0, tek başına , kimse paralel değil

Rol: Tüm sistemin ortak dilini yazar: veri modelleri (`Frame`, `TextBlock`, `Segment`, `TranslationRequest/Result`), soyut arayüzler (`OcrEngine`, `TranslationProvider`), hata taksonomisi ve tip stub'ları.

Gerekli özellikler: Güçlü API tasarımı sezgisi. Değişmezlik (immutability) disiplini — modeller `frozen=True`. İleriye dönük düşünme: `image_crops` alanı bugün yalnızca vision LLM tarafından kullanılıyor ama bugün sözleşmede olmalı, yoksa yarın kırıcı değişiklik olur.

Kabul kriterleri: `mypy --strict` temiz geçer · her model için serileştirme testi · her arayüz için `FakeXxx` referans implementasyonu (diğer ajanlar bununla test yazacak) · sözleşmelerin hiçbirisi somut bir kütüphaneyi (ONNX, Qt, SQLite) import etmez.

Kritik not: Bu ajanın çıktısı **dondurulur**. Değişiklik gerekirse worker ajan doğrudan düzenlemez; orkestratöre *sözleşme değişiklik talebi* açar. Bu kural olmadan çok ajanlı geliştirme çöker.

A2 — Yakalama ve Ekran Ajanı

Model	Sonnet 5
Dizin	<code>src/capture/</code>

Rol: `CaptureService`, monitör listeleme, DPI ölçekleme dönüşümleri, bölge doğrulama, `ChangeDetector` (perceptual hash + kararlılık).

Gerekli özellikler: Windows ekran koordinat sistemi ve per-monitor DPI awareness bilgisi. Bu alanın klasik tuzaklarını bilmek: sanal masaüstü koordinatlarının negatif olabilmesi, monitör takılıp çıkarılması, DPI değişiminde koordinatların kayması.

Kabul kriterleri: $\geq 95\%$ satır kapsamı · yakalama ≤ 10 ms, değişim tespiti ≤ 5 ms (benchmark testi) · DPI %100/%150/%200 dönüşüm testleri · monitör kaybı simülasyon testi · Qt'ye sıfır bağımlılık (headless test edilebilir).

A3 — OCR ve Normalizasyon Ajanı

Model	Sonnet 5
Dizin	<code>src/ocr/</code>

Rol: `RapidOcrEngine` (ONNX Runtime), ön işleme (ölçekleme, kontrast), `TextNormalizer`, ön ayarlar (`dialogue` / `menu` / `tooltip` / `subtitle`).

Gerekli özellikler: ONNX Runtime sağlayıcı yapılandırması (CPU/DirectML). Görüntü ön işlemenin OCR doğruluğuna etkisi. Metin normalizasyonunun dil bağımsız kalması — Türkçe'ye özel varsayım yapmamak.

Kabul kriterleri: OCR ≤ 120 ms (600 px bölge, benchmark) · altın görüntü seti üzerinde normalizasyon regresyon testleri · `TextNormalizer` fonksiyonlarının **tamamı saf** (I/O yok, durum yok) · her ön ayar için ayrı fixture.

Not: Bu ajanın en değerli çıktısı OCR sarmalayıcısı değil, `TextNormalizer`. Çeviri kalitesinin yarısı orada. Görev paketinde bu açıkça vurgulanmalı, yoksa ajan zamanının tamamını OCR ayarına harcar.

A4 — Çeviri Motorları Ajansı

Model	Sonnet 5
Dizin	<code>src/translate/</code>

Rol: `LocalNmtProvider` (CTranslate2), `LocalLLmProvider` (llama.cpp, vision + streaming), `CloudProvider`, prompt şablonları, sözlük/TM enjeksiyonu, düşüş zinciri (fallback chain).

Gerekli özellikler: Prompt mühendisliği — terim sözlüğünü *zorunlu kısıt* olarak, TM örneklerini *few-shot* olarak yerleştirmek. Segment sayısı ile çeviri sayısının hizalı kalmasını garantilemek (LLM'in fazladan açıklama yazma eğilimine karşı savunma). Streaming ve iptal.

Kabul kriterleri: Her sağlayıcı `FakeProvider` ile aynı sözleşme testlerini geçer · `ContractViolation` senaryosu test edilir (LLM fazla/eksik satır döndürdüğünde) · yerel NMT ≤ 150 ms/satır · sözlük terimlerinin çıktıda korunduğunu doğrulayan test · hiçbir prompt veya çeviri içeriği loglanmaz testi · API anahtarının hata mesajlarına sızmadığı testi.

A5 — Kalıcılık ve Hafıza Ajanı

Model	Sonnet 5
Dizin	<code>src/store/</code>

Rol: SQLite şeması ve migration'lar, `TranslationCache` (bellek LRU + kalıcı), `TranslationMemory` (rapidfuzz), `GlossaryStore`, `ProfileStore` (JSON içe/dışa aktarma), `SecretStore` (DPAPI).

Gerekli özellikler: SQLite WAL, indeksleme ve eşzamanlı erişim. Fuzzy arama performansı — TM on binlerce kayda çıktığında `find_similar` yavaşlamamalı. Migration disiplini: şema sürümlenir, kullanıcının TM'si asla kaybolmaz.

Kabul kriterleri: cache `get` ≤ 1 ms · 50.000 kayıtlı TM'de `find_similar` ≤ 20 ms · ileri/geri migration testleri · bozuk veritabanı kurtarma testi (cache yeniden oluşur, **TM korunur**) · profil içe/dışa aktarma round-trip testi · DPAPI şifreleme/çözme testi.

A6 — Pipeline ve Eşzamanlılık Ajanı

Model	Opus 5
Dizin	<code>src/pipeline/</code>

Rol: `PipelineOrchestrator`, aşama tanımları, `maxsize=1` kuyruklar, backpressure, debounce, `seq` tabanlı eskimiş sonuç eleme, iptal token'ları, `QThreadPool` yönetimi, aşama metrikleri.

Gerekli özellikler: Python eşzamanlılık modeli ve GIL davranışı. Qt sinyal/slot thread güvenliği (worker'dan UI'a veri geçişi yalnızca sinyalle). Yarış koşullarını (race condition) tasarımla — kilitle değil — elemek.

Kabul kriterleri: Backpressure testi (frame düşüyor, kuyruk büyümüyor) · debounce testi (yarı yazılmış metin çevrilmiyor) · eskimiş sonuç testi (geç gelen küçük `seq` yok sayılıyor) · iptal testi (izleme durduğunda uçustaki iş temiz iptal ediliyor) · 1000 tick'lik stres testinde bellek büyümüyor · **UI thread'inde hiçbir bloklayıcı çağrı olmadığını doğrulayan test.**

Neden Opus 5: Eşzamanlılık hataları sessizdir, testte görünmez, kullanıcıda rastgele donma olarak ortaya çıkar. Sistemin en pahalı hata yüzeyi burası.

A4 ile paralel çalışır. A6, A4'ün gerçek kodunu beklemesiz; `FakeProvider` ile geliştirir ve test eder. Sözleşme-öncelikli yaklaşımın karşılığı tam olarak budur — pipeline'ın doğruluğu hangi çeviri motorunun takılı olduğuna bağlı değildir.

A7 — UI ve Overlay Ajanı

Model	Sonnet 5
Dizin	<code>src/ui/</code>

Rol: `SnapshotWindow` (donmuş görüntü + seçim etkileşimi), `RegionSelector`, `OverlayStrip` (yapışık + koparılabilir), ayarlar diyalogu, tepsi (tray) ikonu, `HotkeyService`.

Gerekli özellikler: Qt transparan pencere bayrakları (`WA_TranslucentBackground`, `WindowTransparentForInput`, `WindowStaysOnTopHint`). Çoklu monitör + DPI koordinat dönüşümü. Fare olaylarının oyuna geçirilmesi (overlay tıklamayı yutmamalı). Windows `RegisterHotKey` ve çıkışma yönetimi.

Kabul kriterleri: `pytest-qt` ile seçim etkileşimi testleri · overlay'in fare girdisini geçirdiğini doğrulayan test · DPI %100/%150/%200 ve iki monitörde şerit konumlandırma testleri · render ≤ 16 ms · kod hiçbir yerde modal dialog açmaz (statik kontrol testi) · kısayol çıkışması testi.

A8 — Paketleme ve Model Dağıtım Ajanı

Model	Sonnet 5
Dizin	<code>src/models_mgr/</code> , <code>packaging/</code>

Rol: `ModelManager` (indirme, SHA-256 doğrulama, sürüm, devam ettirilebilir indirme), model kayıt defteri (registry), ilk açılış sihirbazı akışı, PyInstaller spec, derleme betiği, üçüncü taraf lisans dosyaları.

Gerekli özellikler: PyInstaller'ın ONNX Runtime / llama.cpp / Qt eklentileri ile bilinen tuzakları (gizli import, ikili dosya toplama). Checksum doğrulama ve kısmi indirme kurtarma. Lisans uyumu (üçüncü taraf lisanslarının pakete dahil edilmesi).

Kabul kriterleri: Temiz Windows sanal makinesinde derlenen paket açılır ve çalışır · yanlış checksum reddedilir ve yeniden indirilir (test) · yarıda kesilen indirme devam eder (test) · `LICENSES/` dizini eksiksiz · exe boyutu ≤ 150 MB (modeller hariç).

A9 — Test Altyapısı ve Fixture Ajanı

Model	Sonnet 5 (koşum), Haiku 4.5 (fixture üretimi)
Dizin	<code>tests/</code>

Rol: Altın görüntü seti (gerçek oyun ekran görüntüleri + beklenen çıktı), `FakeProvider` / `FakeOcrEngine`, performans bütçesi koşumu (`pytest-benchmark`), entegrasyon test iskeleti, CI yapılandırması.

Gerekli özellikler: Fixture'ları küçük ve deterministik tutmak (repo şişmemeli). Performans testlerini makine hızına dayanlı yazmak (mutlak sınır + görelî regresyon birlikte).

Kabul kriterleri: Altın set en az 6 oyun türünü ve 4 ön ayarı kapsar · sahte implementasyonlar A1 sözleşmelerini birebir uygular · performans koşumu bütçe aşımında kırılır · tüm test paketi ≤ 2 dakikada koşar.

Not: Bu ajan Dalga 1'de çalışır, en sonda değil. Sahte implementasyonlar ve fixture'lar olmadan diğer ajanlar test yazamaz.

A10 — İnceleme Ajanı (Reviewer)

Model	Opus 5
Dizin	(hiçbiri — salt okuma, yalnızca rapor yazar)

Rol: Her tamamlanmış modülü sözleşmelere, kabul kriterlerine ve performans bütçesine karşı düşmanca (adversarial) inceler. Kod yazmaz, düzeltmez.

Kontrol listesi:

1. Sözleşmeye uyum: arayüz tam mı, tipler doğru mu, sözleşme dosyaları değiştirilmiş mi?
2. Sahiplik ihlali: ajan kendi dizini dışına yazmış mı?
3. Kanıt: teslim raporundaki test çıktısı gerçek mi, testler gerçekten o davranışı doğruluyor mu, yoksa tautoloji mi?
4. Performans bütçesi: benchmark var mı, geçiyor mu?
5. Hata yolları: §5.6 tablosundaki her satır gerçekten ele alınmış mı?
6. Gizlilik: metin/anahtar loglanıyor mu, diske yazılıyor mu?
7. Sadeleştirme: gereksiz soyutlama, ölü kod, kopyalanmış mantık.

Çıktı: Her bulgu için dosya + satır + neden bozuk olduğu + hangi girdiyle patlayacağı. Karar: **kabul** veya **düzeltilme gerekli**.

Neden Opus 5 ve neden yazmıyor: Kodu yazan ajan kendi kodunu doğru göremez. İnceleme ayrı bir ajan ve daha güçlü bir modelde olmalı. Yazmamasının sebebi de aynı: düzeltmeyi orijinal ajan yapar, çünkü bağlamı onda.

A11 — Dokümantasyon ve Yerelleştirme Ajanı

Model	Haiku 4.5
Dizin	<code>docs/</code> , <code>src/ui/i18n/</code>

Rol: README, kullanıcı kılavuzu (bölge seçme, profil oluşturma, exclusive fullscreen sorun giderme), CHANGELOG, uygulama arayüzü metinleri (TR + EN), sözlük hazırlama rehberi.

Gerekli özellikler: Sade ve doğru Türkçe. Teknik terimleri gereksiz yere Türkçeleştirmemek ama gereksiz İngilizce de bırakmamak. Ekran görüntülerini gerçek uygulamadan almak.

Kabul kriterleri: Kılavuzdaki her adım gerçek uygulamada takip edilerek doğrulanır · TR/EN metin dosyaları anahtar bazında eksiksiz eşleşir (test) · arayüzde sabit kodlanmış metin kalmadığını doğrulayan test.

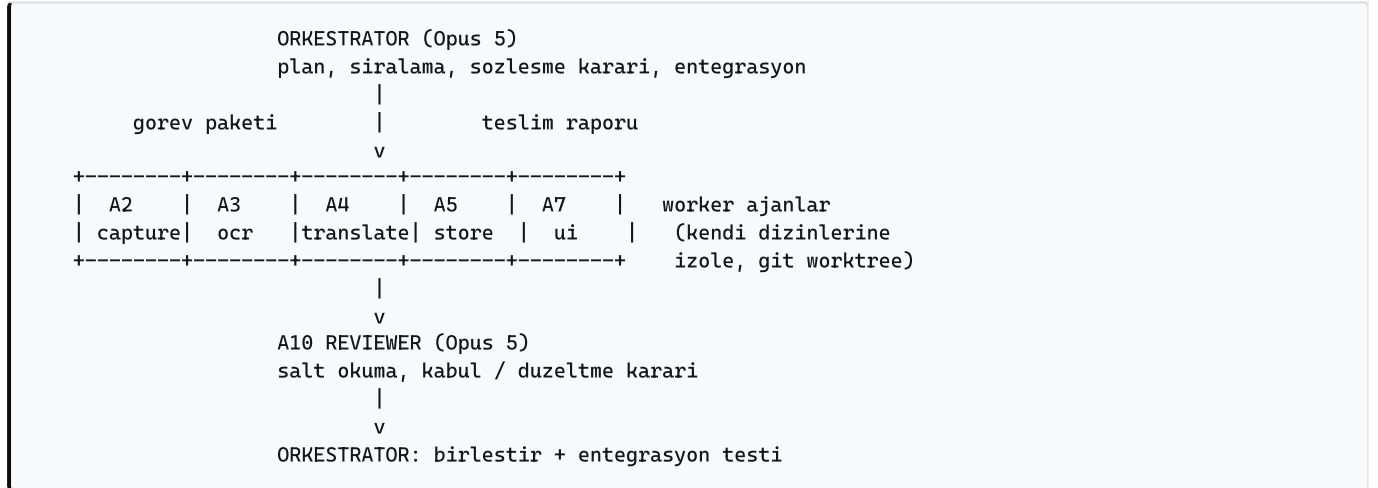
8.4 Model atama gerekçesi

Model	Nereye	Neden
Opus 5	Orkestratör, A1 (sözleşme), A6 (pipeline), A10 (inceleme)	Hatanın geri dönüşü pahalı. Yanlış sözleşme beş ajanın işini çöpe atar; eşzamanlılık hatası testte görünmez; inceleme zayıfsa tüm kalite kapısı çöker.
Sonnet 5	A2, A3, A4, A5, A7, A8, A9	Sınırları net, kabul kriterleri testlerle yazılı, hacimli implementasyon işi. Doğru maliyet/kalite noktası.
Haiku 4.5	A11, A9'un fixture üretimi	Mekanik, düşük muhakeme, yüksek hacim: metin dosyaları, changelog, fixture çoğaltma.

Kural: bir ajan bütçe yüzünden zayıf modele indirilmez. Zayıf modelde yapılan hata, incelemede yakalanır ve tekrar yapılır — bu daha pahalıdır.

8.5 Ajanları birbirine bağlama protokolü

Ajanlar birbirleriyle **doğrudan konuşmaz**. Tüm iletişim orkestratör üzerinden, iki yapılandırılmış belge ile yürür: görev paketi (aşağı doğru) ve teslim raporu (yukarı doğru).



Bağlantı kuralları:

- Sözleşme tek bağlantı noktası.** İki ajanın kodu yalnızca `src/contracts/` üzerinden buluşur. A4 (çeviri), A5'in (store) iç yapısını bilmez; yalnızca arayüzünü bilir.
- Dizin izolasyonu = çakışma yok.** Her ajan kendi git worktree'sinde ve kendi dizininde çalışır. Aynı dosyaya iki ajan asla dokunmaz.
- Sözleşme değişiklik talebi.** Bir worker sözleşmenin eksik olduğunu görürse kodu değiştirmez; talebi teslim raporunda açar. Orkestratör kararı verir, A1 değişikliği yapar, etkilenen tüm ajanlara bildirim gider.
- Kalite kapısı zorunlu.** Teslim → A10 incelemesi → orkestratör kararı. Ret durumunda düzeltme paketi **aynı ajana** gider (yeni ajana değil) — bağlam korunur.
- Entegrasyonu orkestratör yapar.** Hiçbir worker başka bir worker'ın dalını birleştirmez. "Bende çalışıyor" kabul edilebilir bir kanıt değildir.

8.6 Görev paketi şablonu

Her ajana verilen brief bu yapıda olur. Eksik bir alan, ajanın kendi varsayımını uydurmasına yol açar.

GOREV PAKETI - <Ajan kodu> <Ajan adi>
=====

MODEL : <Opus 5 | Sonnet 5 | Haiku 4.5>
DALGA : <0-4>

AMAC : Tek paragrafta ne inşâ edilecek.

SAHIPLIK : Yazabileceğin izinler (mutlak liste).
YASAK : Okuyabilirsin ama YAZAMAZSIN:
- src/contracts/** (dondurulmuş)
- diğer tüm ajan izinleri

SOZLESMELER: Uyman gereken arayüz/tipler (dosya + sınıf adı).

KABUL : Testlerle yazılı kriterler. Her biri kontrolabilir bir komut.
- pytest tests/unit/<alan> -q
- pytest tests/perf/<alan> -q (bütçe assertion'ları)
- mypy --strict src/<alan>

BÜTÇE : Bu modülün performans bütçesi (varsa, ms cinsinden).

YÖNTEM : - TDD: önce başarısız test, sonra implementasyon.
- Yeni bağımlılık = orkestrator onayı gerekir.
- Log'a metin/anahtar yazmak yasak.
- Modal dialog açmak yasak.

TESLİM : Teslim raporu şablonunu (§8.7) eksiksiz doldur.
Test çıktılarını YAPISTIR. Kanıt yoksa iş bitmemiştir.

8.7 Teslim raporu şablonu

TESLİM RAPORU - <Ajan kodu>
=====

DURUM : tamamlandı | kısmi | bloke

YAPILAN : Madde madde, dosya yollarıyla.

TEST KANITI : Kusulan komutlar ve GERÇEK çıktıları (yapıştırılmış).
Özet değil, çıktının kendisi.

BÜTÇE OLCUMU : Asama basına ölçülen ms değerleri ve bütçeyle karşılaştırma.

SOZLEŞME : Sapma var mı? Değişiklik talebi var mı (gerekçe ile)?

BİLİNEN EKSIK: Bilerek yapılmayanlar ve nedenleri.

SONRAKİ AJAN : Bu modülü kullanacak ajanın bilmesi gerekenler.

RISK : Fark ettiğin ama kendi kapsamında olmayan sorunlar.

8.8 İnşa dalgaları

Dalga	Ajanlar	Paralel mi	Çıkış kriteri
0	A1 sözleşmeler	Hayır — tek başına	<code>mypy --strict</code> temiz, sahte implementasyonlar hazır, sözleşme donduruldu
1	A2 capture · A3 ocr · A5 store · A9 test altyapısı	Evet, 4 paralel	Her modül kendi testleriyle bağımsız geçiyor; A10 dördünü de kabul etti
2	A4 translate · A6 pipeline	Evet, 2 paralel	Sahte sağlayıcı ile uçtan uca pipeline testi geçiyor (UI yok)
3	A7 ui	Hayır	Mod 2 önce çalışır hale gelir, sonra Mod 1. Gerçek ekranda elle doğrulama.
4	A8 paketleme · A11 dokümantasyon	Evet, 2 paralel	Temiz VM'de paket açılıp çalışıyor; kılavuzun her adımı doğrulandı

A10 (Reviewer) her dalganın sonunda devreye girer; dalga kapanmadan sonraki dalga başlamaz.

Neden Mod 2 önce: Mod 2, pipeline'ın tamamını (yakalama → değişim → OCR → çeviri → render) sürekli çalıştırır. Çalışıyorsa Mod 1 kolaydır — Mod 1 aynı parçaların tek seferlik ve daha basit bir dizilimidir. Tersı sırada, Mod 1 çalışırken pipeline'ın canlı davranışı hiç sınanmamış olur.

8.9 Tüm ajanlar için ortak zorunlu kurallar

Her görev paketine değişmeden kopyalanır:

- TDD.** Önce başarısız test, sonra implementasyon. Test yazılmadan yazılan kod teslim edilemez.
- Sözleşmeye dokunma.** `src/contracts/` salt okunur. Eksik varsa talep aç.
- Dizinini terk etme.** Sahip olmadığın dizine tek satır yazmak sahiplik ihlalidir ve teslim reddedilir.
- Kanıt yapıştır.** Testleri gerçekten çalıştır, çıktıyı rapora koy. "Testler geçiyor" cümlesi kanıt değildir.
- Bütçeyi aşma.** Performans bütçesini aşan kod teslim edilemez; aşıyorsa raporda açıkça belirt.
- Bağımlılık için onay al.** Yeni paket = lisans + boyut + PyInstaller uyumu değerlendirmesi.
- Loglama disiplini.** OCR metni, çeviri içeriği, API anahtarı asla loglanmaz.
- Modal dialog yok.** Hata durum çubuğuna yazılır; kullanıcının ekranı kesilmez.
- Dürüst raporla.** Bir şey çalışmıyorsa çalışmıyor yaz. Kısmi teslim kabul edilebilir; yanlış rapor edilemez.

9. Test ve Kalite Stratejisi

Katman	Kapsam	Araç
Birim	Saf fonksiyonlar: normalizasyon, hash, koordinat dönüşümü, cache anahtarı	pytest
Sözleşme	Her OcrEngine ve TranslationProvider implementasyonu aynı test paketini geçer	pytest parametrize
Altın set	Gerçek oyun ekran görüntüleri → beklenen OCR/normalizasyon çıktısı	pytest + fixture
Entegrasyon	Sahte sağlayıcılarla uçtan uca pipeline (ağ ve model olmadan)	pytest
Eşzamanlılık	Backpressure, debounce, eskimiş sonuç, iptal, bellek sızıntısı	pytest + stres koşumu
UI	Seçim etkileşimi, DPI, çoklu monitör, girdi geçirme	pytest-qt
Performans	Aşama bütçesi assertion'ları	pytest-benchmark
Elle	Gerçek oyunlarda doğrulama listesi (en az 3 oyun, 3 tür)	kontrol listesi

Değişmez kural: **performans bütçesi aşımı testi kırar**. Bütçe bir gözlem değil, bir sözleşmedir.

10. Riskler ve Azaltımlar

Risk	Etki	Azaltım
OCR stilize oyun fontlarında başarısız	Yüksek	Ön ayarlar + ön işleme; başarısız durumda Mod 1 ile manuel seçim; altın sete zor örnekler eklenir
Yerel LLM zayıf makinede çalışmaz	Yüksek	NMT her zaman mevcut ve varsayılan; LLM opsiyonel. Donanım tespiti ile otomatik öneri
Türkçe çeviri kalitesi yetersiz	Yüksek	Katman 0 (sözlük + TM) baştan var; Katman 3 yol haritasında; motor seçimi altın set ölçümüyle yapılır
PyInstaller + ONNX/llama.cpp paketleme sorunları	Orta	A8 Dalga 1'de bir "duman testi" paketi üretir; sona bırakılmaz
Exclusive fullscreen kullanıcıyı hayal kırıklığına uğratar	Orta	Tespit + net yönlendirme + koparılabılır şerit çıkış yolu
Python GIL nedeniyle UI takılması	Orta	A6 Opus 5'te; UI thread'inde bloklayıcı çağrı olmadığını doğrulayan test
Model indirmeleri büyük, ilk deneyim yavaş	Orta	Aşamalı indirme: önce OCR + NMT (~1 GB) ile çalışır hale gel, LLM sonra
Çok ajanlı geliştirmede sözleşme kayması	Orta	Sözleşme dondurma + tek elden değişiklik + A10 sahiplik ihlali kontrolü
Topluluk profilleri lisans/telif sorunu	Düşük	Profiller yalnızca ayar ve terim içerir; oyun metni veya varlığı içermez

11. Yol Haritası

- v1 — Temel** (*bu doküman*) İki mod · yerel OCR · Katman 0/1/2 · yapışık + koparılabilir şerit · oyun profilleri · opsiyonel bulut
- v1.1 — Cilalama** Daha fazla OCR ön ayarı · profil paylaşım biçimi ve içe aktarma UI'ı · DXGI Desktop Duplication ile daha hızlı yakalama · toplu sözlük düzenleme
- v2 — Yerinde kaplama** Çevirinin orijinal metnin üzerine binmesi: arka plan rengi örnekleme, metin sığdırma/küçültme, animasyonlu arka planda titreme önleme
- v2.x — Katman 3: kendi çeviri modeli** (*ayrı spec*) Veri toplama ve hizalama · damıtma koşumu · QLoRA eğitimi · kalite ölçüm koşumu · **FineTunedProvider**
- Değerlendirilecek (taahhüt değil)** Manga/dikey metin okuma sırası · TTS · macOS
-

12. Açık Sorular

Bunlar v1 planını bloke etmez; ilgili dalga başlamadan önce ölçümle kapatılır.

- Yerel LLM modeli hangisi?** Gemma 3 4B ve Qwen3 4B sınıfı adaylar, altın set üzerinde TR çıktı kalitesi + hız + VRAM açısından ölçülüp seçilecek. Karar A4'ün görev paketinden önce verilir.
 - NMT: NLLB-600M mi, dil çifti başına OPUS-MT mi?** Tek büyük model (tek indirme, tüm diller) ile çok küçük model (daha hızlı, daha isabetli, dil başına indirme) arasındaki takas ölçümle kapatılır.
 - Yakalama:** **mss** v1 için yeterli mi? 10 ms bütçesi tutuyorsa evet. Tutmuyorsa DXGI Desktop Duplication v1'e çekilir. A2'nin ilk benchmark'ı karar verir.
 - Uygulama ikonu.** İsim karara bağlandı (Suflör); ikon hâlâ gerekli. A8 (paketleme) ve A11'i etkiler, Dalga 4 öncesinde lazım. Tiyatro/kulis temasından türetilbilir.
 - Dağıtım kanalı.** GitHub Releases yeterli mi, yoksa imzalı yükleyici (kod imzalama sertifikası) gerekiyor mu? SmartScreen uyarısı kullanıcı güvenliğini etkiler.
 - Lisans.** Depo herkese açık ama lisans dosyası henüz yok — bu durumda yasal varsayılan "tüm hakları saklı"dır. Katkı kabul edilecekse veya kullanıcıların dağıtması istenirse açık bir lisans (MIT/Apache-2.0) seçilmeli. PySide6'nın LGPL olması MIT uygulama koduna engel değil (dinamik bağlama).
-

Karar Kaydı

Bu tasarımda kilitlenen kararlar ve gerekçeleri:

Karar	Seçim	Gerekçe
Uygulama adı	Suflör	Kuliste repliği fıslıdayan kişi; "oyunu kesmeden kenardan anlamı ver" ürün ilkesini birebir taşıyor
"Ajan" tanımı	Kodu yazacak geliştirme ajanları	Kullanıcı kararı; runtime mimari ayrı bölümde bileşen olarak modellendi
Yığın	Python 3.12 + PySide6	En zengin OCR/AI ekosistemi, tek dil, en hızlı prototip
Hedef içerik	Oyun öncelikli + genel destek	Mimari genel, oyun senaryosu birinci sınıf
Motor stratejisi	Mod bazlı hibrit	Mod 1 seyrek/kaliteli, Mod 2 sürekli/hızlı — farklı motor gerektirir
Maliyet duruşu	Ücretsiz + offline varsayılan, bulut opsiyonel	Sıfır sürtünmeli ilk deneyim; kullanıcı anahtar girmeye zorlanmaz
Gösterim	Yapışık şerit + koparılabilir pencere	Metin taşma/sığdırma sorunu yok, v1'de güvenilir çalışır
Pipeline	Tek process, 5 aşama, bounded queue + backpressure	UI donmasını çözer; IPC bedelini ödemedi ayrı process dikişini açık bırakır
Anti-cheat	Sıfır injection	Oyun hesabı riski kabul edilemez
Sözleşme yönetimi	<code>src/contracts/</code> dondurulur, tek elden değişir	Çok ajanlı paralel geliştirmenin tek şartı